

webDevAgent

Autonomous AI Workflow for Website Development and
Deployment

Name : Soumik Roy
Framework: MachinaOS
Date: July 13, 2026

Contents

Abstract	2
1 Introduction	3
1.1 Objectives	3
2 System Design	4
2.1 Architecture	4
3 Workflow	5
3.1 Execution Flow	5
4 Components	6
5 Design Decisions	7
6 User Guide	8
7 Demo and Screenshots	9
8 FAQ	11
9 Conclusion	12

Abstract

webDevAgent is an autonomous multi-agent workflow that converts natural language requirements into deployed web applications. The workflow combines a Coding Agent responsible for software development tasks with a Web Agent responsible for GitHub operations and deployment. Together they automate the complete website development lifecycle.

Chapter 1

Introduction

webDevAgent was developed to reduce manual effort during website development. Instead of switching between editors, terminals, browsers and GitHub, users provide a requirement and the workflow performs the remaining tasks autonomously.

1.1 Objectives

- Generate complete web applications.
- Modify existing projects.
- Manage files and execute shell commands.
- Push changes to GitHub.
- Trigger deployment through Vercel-connected repositories.

Chapter 2

System Design

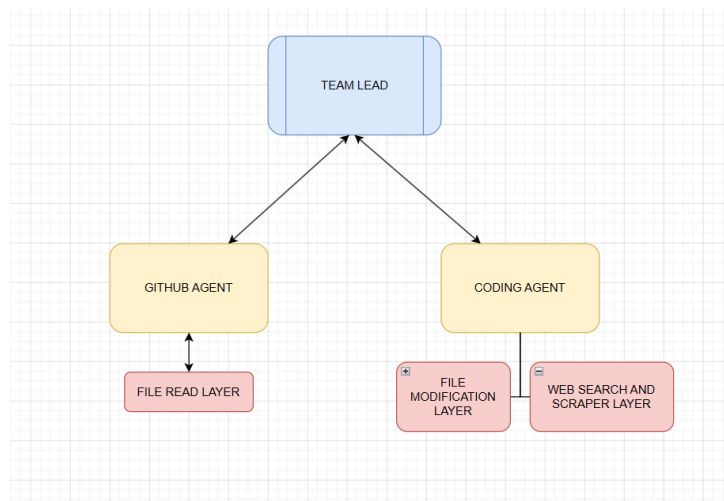


Figure 2.1: Overall Architecture

2.1 Architecture

The workflow consists of three major components:

1. AI Employee
2. Coding Agent
3. Web Agent as Github Agent

The AI Employee acts as the orchestrator. Development related tasks are assigned to the Coding Agent while GitHub operations, commits, pushes and deployment are delegated to the Web Agent.

Workflow

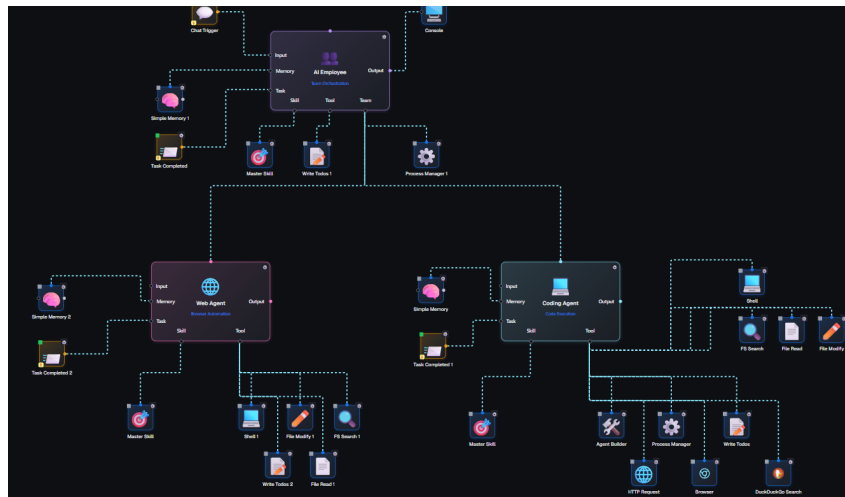


Figure 3.1: Workflow Overview

3.1 Execution Flow

1. User submits a website request.
2. AI Employee analyses the request.
3. Coding tasks are routed to the Coding Agent.
4. GitHub related tasks are routed to the Web Agent.
5. Coding Agent generates or edits the project.
6. Web Agent commits changes and deploys the application.
7. Live website URL is generated on your vercel account.

Chapter 4

Components

Component	Responsibility
AI Employee	Task planning and orchestration
Coding Agent	Code generation, editing, debugging, shell execution, file management, search and HTTP requests
Web Agent	GitHub repository management and deployment
Memory	Maintains workflow context
Shell	Executes terminal commands
Browser	Browser automation and verification

Chapter 5

Design Decisions

Why Two Agents?

- Separation of responsibilities
- Better maintainability
- Easier debugging
- Independent scaling
- Improved security for repository access

Advantages

- Fully autonomous workflow
- Modular architecture
- Reusable components
- Automatic deployment
- Easy integration of future agents

Chapter 6

User Guide

Prerequisites

- MachinaOS installed
- GitHub account connected
- Repository connected to Vercel
- Required API credentials

Usage

1. Launch the workflow.
2. Provide a website requirement.
3. Wait while the Coding Agent develops the project.
4. The Web Agent commits and deploys the application.
5. Access the generated website through the deployment URL.

Chapter 7

Demo and Screenshots

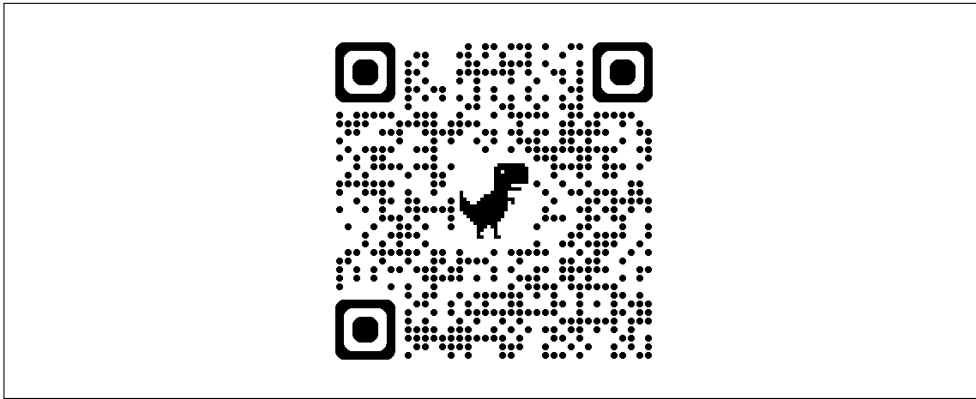


Figure 7.1: Demo Video, Here's the Link: [Click here](#)

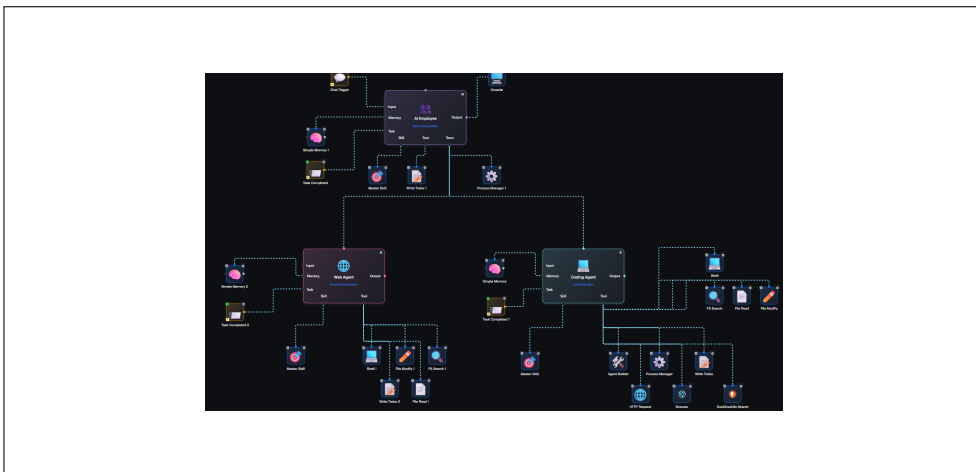


Figure 7.2: Workflow Screenshot

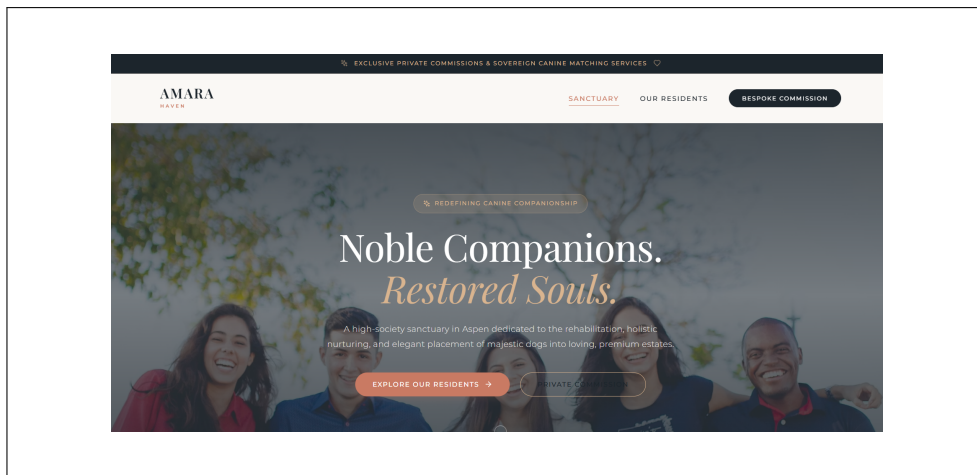


Figure 7.3: Deployment Output

Chapter 8

FAQ

1. Why use two agents instead of one?

Ans: To distribute the task and for easier debugging

2. Can existing projects be modified?

Ans: Yes, this can be done first you need to provide the files in the working directory and then ask the AI to review them and then to make the changes

3. How is deployment automated?

Ans: To make deployment easier I already have connected repos to vercel.

If you wish you can directly skip github and provide the personal tokens credentials(of vercel or some other hosting site) to the AI Agent and you can deploy it there directly.

4. Can additional deployment providers be integrated?

Yes, refer to the previous answer

5. What happens if deployment fails?

Ans: Yes it's rare but has a chance to happen, incase if it happens :

1. Don't panic
2. Check the terminal log, try to look what's going wrong, then try to figure out how you can rectify it

Chapter 9

Conclusion

webDevAgent demonstrates how a modular multi-agent architecture can automate the complete software development lifecycle. By separating implementation and deployment responsibilities, the workflow remains scalable, maintainable and easy to extend.

Future Work

- Docker deployment
- Automated testing
- Security auditing agent
- Monitoring and analytics